

Replacing Heuristic Rule Ordering in Symbolic Integration with Learned Policies

Rohit John

MComp Computer Science
The University of Bath
2025/26

Replacing Heuristic Rule Ordering in Symbolic Integration with Learned Policies

Submitted by: Rohit John

Copyright

Attention is drawn to the fact that copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the author unless otherwise specified below, in accordance with the University of Bath's policy on intellectual property (see https://www.bath.ac.uk/publications/university-ordinances/attachments/Ordinances_1_October_2020.pdf).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

Symbolic integration remains one of the most challenging problems in computer algebra, lacking a single complete algorithmic solution. Rule-based solvers such as SymPy's `manualintegrate` rely on hand-crafted heuristics to select transformation rules, often resulting in long, inefficient derivations. This dissertation investigates learned policies for guiding rule selection within the `manualintegrate` framework, combining the rigour of a computer algebra system with the learning capabilities of neural networks. Two models are trained and evaluated: a supervised behaviour-cloning classifier trained on two million labelled rule-demonstration pairs, and a PPO actor-critic agent trained on fifty thousand unlabelled integration episodes with no per-step labels. Both models reduce unnecessary search without sacrificing the correctness guarantees provided by symbolic reasoning, with the RL agent achieving a 80% reduction in runtime and a 70% reduction in rule attempts relative to `manualintegrate`, while also improving accuracy and resolving expressions the heuristic cannot. These results provide evidence that reinforcement learning is a principled and effective approach to improving the efficiency and capability of computer algebra systems.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem statement	2
1.3	Hypotheses	3
1.4	Contributions	3
2	Literature and Related work	4
2.1	Symbolic Integration and Computer Algebra	4
2.2	Machine Learning for Symbolic Integration	5
2.2.1	Lample & Charton	5
2.2.2	SIRD	5
2.2.3	AlphaIntegrator	6
2.3	Reinforcement Learning for Computer Algebra	6
3	Technical background	8
3.1	Reinforcement Learning Foundations	8
3.2	Sequence Encoders and Mathematical Representation	9
4	Methodology	11
4.1	MDP Formulation for Rule Selection	11
4.2	The Supervised Rule-Selection Model	12
4.2.1	Architecture	12
4.2.2	Training Data: SIRD	12
4.2.3	Training Objective	13
4.2.4	Inference	13
4.3	The Reinforcement Learning Agent	13
4.3.1	Architecture	13
4.3.2	Training Algorithm	13
4.4	Methodological choices	15
4.4.1	Ranking Versus Sampling	15
4.4.2	Reward Shaping	15
4.5	Datasets	15
4.5.1	Training Data	15
4.5.2	Training Distributions	16
4.5.3	Evaluation Distributions	16
4.5.4	Train/Validation/Test Splits	16
4.6	Evaluation Protocol	17

4.6.1	Metrics	17
4.6.2	Baselines	17
5	Implementation and Experimental Setup	18
5.1	Environment Design	18
5.1.1	PPO Implementation	18
5.1.2	Modifying manualintegrate	18
5.1.3	Episode Structure	19
5.1.4	Timeout Handling	19
5.2	Supervised Model Training	19
5.2.1	Data Pipeline	19
5.2.2	Training and Convergence	19
5.3	RL Agent Training	19
5.3.1	Rollout Collection and Reward Assignment	19
5.3.2	Advantage Estimation and PPO Update	20
5.4	Engineering Challenges	20
5.4.1	Unintended Training Behaviour	20
5.4.2	Training Time	20
6	Results and Discussion	21
6.1	FWD, IBP, and BWD Datasets	21
6.1.1	In-distribution performance: FWD	22
6.1.2	Out-of-distribution generalisation: IBP	22
6.1.3	Out-of-distribution generalisation: BWD	22
6.1.4	Supervised model comparison	22
6.2	Branching Efficiency Analysis	23
6.3	External Validation: RUBI Independent Test Suite	24
6.3.1	Performance on RUBI	24
6.4	Advanced Solving Examples	25
7	Limitations and Future Work	26
7.1	Limitations	26
7.2	Future Work	26
8	Summary and Conclusions	28
	Bibliography	29
A	Full Hyperparameter Tables and Training Curves	33
A.1	Software Environment	33
A.2	Shared Transformer Architecture	33
A.3	Supervised Model Training	34
A.4	RL Agent Training	34
A.5	Training Curves	35
A.6	Integration Rules	36

List of Figures

1.1	Two examples of <code>manualintegrate</code> search tree from (Sharma, Nagpal and Balin, 2023).	2
4.1	Interaction between RL Agent and SymPy environment during training . . .	14
6.1	Distribution of branch counts on the FWD evaluation set. RL produces a tighter, lower-mass distribution than either SUPERVISED or MANUAL. . . .	23
6.2	Branch-count distributions: RL vs MANUAL on the FWD dataset.	24
6.3	Branch-count distributions: RL vs SUPERVISED on the FWD dataset. . . .	24
A.1	Supervised training loss over 15 epochs	35
A.2	Policy & value loss & episode reward over 1000 epochs of training	35

List of Tables

6.1	Integration results across FWD, IBP, and BWD datasets.	21
6.2	Performance on the RUBI independent test suite.	24
6.3	Integrands solved completely by RL but only partially by MANUAL.	25
A.1	Shared Transformer encoder hyperparameters.	33
A.2	Supervised model training hyperparameters.	34
A.3	RL agent training hyperparameters.	34
A.4	Integration rules used by the solver.	36

Acknowledgements

I would like to thank Dr. Rashid Barket for suggesting this topic and providing invaluable support and encouragement during its development, my supervisor Dr. Andreas Theophilou for his suggestions and feedback and my family and friends for their support throughout this work.

This project was primarily developed in the Temple lab at the University of Bath and my thanks goes to its occupants for providing feedback and company during long days.

Generative AI was utilised in the technical development of this project and proofreading the dissertation.

Chapter 1

Introduction

1.1 Motivation

Computer algebra systems (CAS) such as Mathematica, Maple, and SymPy are foundational tools in science and engineering, used daily to perform exact symbolic computation including algebraic manipulation, differential equations, and closed-form integration. Unlike numerical methods, which approximate a result to a given precision, a CAS operates exactly: it either produces a provably correct symbolic answer or reports that no such answer could be found. This exactness is crucial in environments where interpretability, further symbolic manipulation, or rigorous derivation are required.

Symbolic integration, the computation of an antiderivative $F(x)$ such that $F'(x) = f(x)$, is among the most common operations a CAS is asked to perform. Unlike symbolic differentiation, which is a mechanical application of a small set of rules, symbolic integration has no single complete algorithm for the full class of elementary functions. The Risch algorithm (Risch, 1969, 1970) provides a decision procedure for whether an elementary antiderivative exists, but its full implementation is an ongoing research project. Rather than relying on a single complete algorithm, modern systems such as Mathematica's `INTEGRATE`, Maple's `INT`, and SymPy's `INTEGRATE` utilise a range of partial methods: pattern-matching rule sets, heuristic extensions of the Risch algorithm, and lookup tables (Bronstein, 2005; Meurer et al., 2017).

SymPy is one of the most widely used CASs and offers a rule-based integrator called `manualintegrate` (Meurer et al., 2017). Rather than attempting to output an antiderivative directly, `manualintegrate` imitates by-hand integration, applying integration rules such as the power rule, substitution rule, or integration by parts in a sequence of transformation and reduction steps to arrive at an answer. Figure 1 shows two examples of the `manualintegrate` search tree.

This design has two advantages: it produces human-readable step-by-step derivations, and it guarantees correctness by construction because every rule application is verified symbolically. However, it has a significant efficiency limitation: the 24 available rules are attempted in a fixed, hand-coded heuristic order at every step. This ordering was designed for simplicity and stability, and does not adapt to the integrand being processed.

Because `manualintegrate` operates recursively, each sub-integral independently triggers the full fixed rule-trial sequence. The total cost compounds across the integration tree: when the

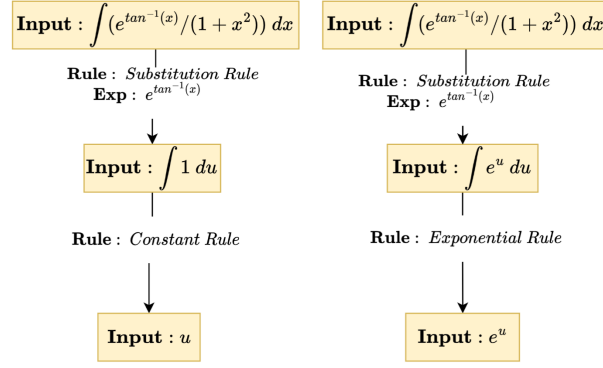


Figure 1.1: Two examples of manualintegrate search tree from (Sharma, Nagpal and Balin, 2023).

correct rule for a sub-expression lies near the end of the fixed ordering, every node pays the cost of trialling all preceding rules first. Additionally, certain rules succeed in transforming a sub-expression but produce sub-problems that ultimately cannot be resolved, leading to backtracking and wasted downstream computation. A policy that consistently attempts the correct rule earlier in the ordering could avoid these false-path rules and recover a large fraction of this wasted computation.

With the development of Deep Learning (DL) and its application to Reinforcement Learning (RL), AI systems have achieved strong performance on complex sequential decision-making problems. While many LLMs have previously struggled with solving mathematical problems, AlphaGeometry (Trinh et al., 2024) and AlphaProof (Hubert et al., 2025) recently achieved medal-level performance on the International Mathematical Olympiad, outperforming many of the best students in the world by modelling the proof process as a sequential decision-making process and using formal verification systems, with the latter using RL to guarantee correctness throughout the proof.

We take a similar approach to improving manualintegrate’s rule ordering. By training two models, a supervised behaviour-cloning classifier and a reinforcement-learning agent to replace manualintegrate’s fixed rule ordering, we aim to improve integration efficiency.

1.2 Problem statement

We formalise the rule-ordering problem as follows. Let $\mathcal{R} = \{r_1, r_2, \dots, r_{24}\}$ denote the set of integration rules available in manualintegrate. Given an integrand $f(x)$, integration proceeds as a sequence of rule-selection decisions over a tree of sub-expressions. At each node s_t representing a sub-expression requiring integration, the solver selects a rule $a_t \in \mathcal{R}$ to apply. Rules are attempted in order until the first applicable rule is found, or all rules are exhausted. A successful rule application either resolves the sub-expression directly or decomposes it into one or more child sub-expressions, each of which becomes a new node. The episode terminates when all sub-expressions have been resolved (success), when all rules are exhausted at some node (failure), or when a maximum rule-attempt limit is reached.

The objective is to learn a policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{R})$ mapping the current sub-expression to a distribution over rules such that rules are ranked in an order that minimises the expected total number of rule attempts across the integration tree, subject to the constraint that the final

output is a correct antiderivative. Correctness is guaranteed by the underlying SymPy engine: the policy influences only the order in which rules are tried while leaving rule application to SymPy.

1.3 Hypotheses

This dissertation investigates three hypotheses.

H1 - Learned ordering outperforms the heuristic . A learned rule-ordering policy, trained on integration examples, can outperform `manualintegrate`'s fixed hand-coded ordering on solve time and rule-attempt count, without sacrificing integration accuracy. This hypothesis asserts that the information contained in an integrand's syntactic structure is sufficient to predict, better than the heuristic, which rules are likely to succeed, and that a neural network can extract this information.

H2 - RL is competitive with supervised learning at lower data cost. A reinforcement learning agent, trained with reward signals and no labelled rule demonstrations can match or exceed the performance of a supervised behaviour-cloning model trained on the same architecture, while requiring substantially fewer training examples.

H3 - Learned orderings generalise across integrand distributions. The performance advantages observed when training and evaluating on the same integrand distribution continue when evaluating on integrands drawn from different distributions and from an independent real-world test suite. This hypothesis tests whether the learned policies capture structural properties of integrands or merely overfit to the statistics of the training generator.

1.4 Contributions

This dissertation makes the following contributions.

- **First RL-based rule-selection policy for symbolic integration.** We introduce the first reinforcement learning approach to rule ordering within SymPy's `manualintegrate` framework. A PPO actor-critic agent, using an encoder-only Transformer over prefix-notation expression representations, learns to rank the 24 candidate rules from environment interaction alone, requiring no labelled rule demonstrations.
- **An extended supervised integration model.** We implement and comprehensively evaluate a supervised rule-selection model trained on the SIRD dataset (Sharma, Nagpal and Balin, 2023), using a previously unexplored architecture and improving prior work.
- **Extended evaluation across datasets and metrics.** We evaluate all three methods on four datasets spanning in-distribution, out-of-distribution, and real-world textbook integrals (RUBI), the last of which has not previously been used to evaluate learned rule-selection policy. In addition to the accuracy metric, we report rule-attempt count, branch count, and runtime as complementary efficiency metrics, providing a more complete picture of search cost than single-dataset accuracy alone.

Chapter 2

Literature and Related work

2.1 Symbolic Integration and Computer Algebra

Systematic study of symbolic integration dates to the early nineteenth century (Laplace, 1820) and evolved from analytic calculus into an algebraic discipline with Liouville's theorem (Rosenlicht, 1972), which established the strict form any elementary antiderivative must take and enabled the development of algorithmic decision procedures .

Building on this foundation, Risch (1969, 1970) published the Risch algorithm, a decision procedure which determines whether a given elementary function possesses an elementary indefinite integral and computes it explicitly if so, by transforming the problem into polynomial and rational-function manipulation (Risch, 1969, 1970; Bronstein, 2005; Geddes, Czapor and Labahn, 1992).

Despite its theoretical completeness, a full software implementation of the Risch algorithm has proven exceptionally difficult due to the over a hundred pages of formal description, requiring complex machinery to manage overlapping algebraic field extensions and has never been fully implemented in any production CAS (Geddes, Czapor and Labahn, 1992; Trager, 1984).

Due to these implementation barriers, developers of early computer algebra systems such as MACSYMA and REDUCE implemented heuristic and semi-algorithmic alternatives that sacrifice formal completeness for practical speed. (Norman and Moses, 1976) extended CAS integration to handle special function classes defined in terms of integrals, broadening the coverage of heuristic solvers to encompass identities common in mathematical physics. (Fitch, 1981) further optimised the REDUCE integration system, explicitly prioritising computation speed and recognisable output forms over strict adherence to a theoretical decision procedure.

Building on this work, Arthur Norman developed a faster variant known as the Risch–Norman parallel integration algorithm, which avoids constructing the full tower of differential extensions required by the original Risch algorithm (Bronstein, 1987) and now underpins the integration routines in commercial systems including Maple's `int` and Mathematica's `Integrate`.

Despite these algorithmic advances, many modern open-source computer algebra systems such as SymPy (Meurer et al., 2017) continue to rely on heuristic, rule-based solvers for interactive use, but as discussed in Section 1, can use inefficient searches to determine valid rules.

Currently the most extensive rule-based integral solver is RUBI (Rule-Based Integrator; Rich,

Scheibe and Abbasi 2018), which organises more than 6,700 integration rules as a structured decision tree with carefully specified applicability conditions. RUBI’s rule set substantially exceeds the 24 rules of `manualintegrate` and is benchmarked against a test suite of over 72,000 integrals drawn from textbooks and research papers. A subset of 1,832 problems from this dataset serves as an out-of-distribution evaluation set in Chapter 5.

2.2 Machine Learning for Symbolic Integration

2.2.1 Lample & Charton

The first demonstration that a neural network could perform symbolic integration was provided by (Lample and Charton, 2020), who trained a sequence-to-sequence Transformer to predict antiderivatives directly from the integrand, treating integration as a translation problem. The model achieved high accuracy on the within-distribution datasets and compared favourably to commercial CAS tools. In addition to establishing the direct-prediction approach, (Lample and Charton, 2020) introduced the prefix-notation tokenisation scheme for mathematical expressions used throughout subsequent work, including this dissertation, and the data generation procedures that provide the training and evaluation distributions for both learned models presented here (Section 4.5).

While this approach appeared promising, (Welleck et al., 2022) demonstrated significant weaknesses in (Lample and Charton, 2020)’s implementation. Using a genetic algorithm to construct failure cases, they showed that sequence-to-sequence integrators trained on the Lample–Charton distributions fail consistently under minor algebraic perturbations that preserve the identity of the integrand.

2.2.2 SIRD

The most direct supervised precedent for this work is the Symbolic Integration Rules Dataset (SIRD) of (Sharma, Nagpal and Balin, 2023). Rather than predicting antiderivatives directly, SIRD trains a sequence-to-sequence encoder-decoder Transformer to predict at each integration state, the rule `manualintegrate` would apply. For more complex rules such as `substitution_rule`, the model outputs both the rule name and the relevant sub-expression (e.g., `[substitution_rule, exp, atan, x]`).

The dataset was constructed by running `manualintegrate` over generated integrands and recording each rule and application expression at every step, yielding approximately two million labelled (integrand state, rule) pairs. The supervised model in this dissertation diverges from the original SIRD architecture in two ways: it uses an encoder-only classifier rather than an encoder-decoder, and it predicts only the rule index without any application expression, allowing them to be determined by SymPy’s internal logic. This simplifies the prediction task and removes the decoder component while retaining the core guided-ordering benefit.

While SIRD demonstrates improvement over the heuristic baseline, it has two structural limitations that motivate our work.

First, where multiple valid rules exist for a given sub-expression, SIRD records only the single path the heuristic happened to take. The supervised model therefore never observes alternative correct orderings, whereas the RL agent, optimising over complete episode outcomes, can discover more efficient trajectories that the labelled dataset does not contain.

Second, SIRD adapts the Lample & Charton encoder-decoder architecture to rule prediction without architectural modification, achieving a marginal improvement over the manual integrate baseline, with a 4 percentage point improvement on in-domain test sets, and a decline on out-of-domain problems. An encoder-only classifier reduces the prediction task to a single forward pass and removes the decoder’s sequential generation overhead, which is unnecessary for rule classification.

2.2.3 AlphaIntegrator

(Ünsal, Gehr and Vechev, 2024) developed AlphaIntegrator, a similar system for step-by-step symbolic integration. Rather than predicting antiderivatives directly, it trains a small decoder-only GPT-style Transformer to propose at each subexpression, an action from a custom parametric action space and any required action parameters. These proposed actions are executed by their own symbolic engine.

The model is trained on 42.5M synthetically generated integration steps using standard cross-entropy loss and deployed with beam search, achieving 87.3% accuracy against SymPy’s 83.3% on a held-out test set while exploring approximately 50% fewer search nodes.

AlphaIntegrator has several limitations that motivate the present work. Evaluation is confined to a single in-distribution test set derived from the same FWD generator used for training; no out-of-distribution evaluation is reported, leaving generalisation under distribution shift uncharacterised. Similar to (Sharma, Nagpal and Balin, 2023), the accuracy improvement over SymPy is modest at 4 percentage points, and runtime comparisons between AlphaIntegrator and SymPy are not reported, making it difficult to assess practical efficiency gains beyond the node-exploration metric.

2.3 Reinforcement Learning for Computer Algebra

One of the most influential early applications of reinforcement learning to combinatorial search was demonstrated by AlphaGo (Silver et al., 2016) and AlphaZero (Silver et al., 2017). AlphaGo combined a learned policy and value network with Monte Carlo Tree Search to achieve superhuman performance in Go; AlphaZero generalised this framework to Chess and Shogi through self-play from random initialisation and without human demonstration data.

(Polu and Sutskever, 2020) and (Polu et al., 2022) transferred this approach to formal theorem proving. Their GPT-f system trained a language model to generate proof steps accepted by the Lean and Metamath proof assistants, and subsequent work introduced a curriculum learning procedure in which proof difficulty is gradually increased during training, yielding substantial performance improvements. (Bansal et al., 2019) developed HOList, an environment for ML-guided higher-order logic theorem proving in HOL Light, and demonstrated that RL improved over supervised imitation learning on the same task.

The closest RL based work to this dissertation is (Peifer, Stillman and Halpern-Leistner, 2020), who applied PPO to S-pair selection in Buchberger’s algorithm for computing Gröbner bases. Buchberger’s algorithm involves a discrete selection at each step whose optimal choice is not analytically known, the total computational cost depends on the full sequence of selections, and the algebraic engine guarantees correctness regardless of the order chosen. (Peifer, Stillman and Halpern-Leistner, 2020) demonstrated reductions of 20–40% in total polynomial additions

compared to state-of-the-art hand-crafted heuristics across several distributions.

(Jia et al., 2023) applied RL to variable ordering in Cylindrical Algebraic Decomposition (CAD), where the doubly-exponential worst-case complexity makes ordering decisions particularly consequential. Their graph neural network policy, trained with a policy gradient method, consistently outperformed heuristic-based orderings and demonstrated the utility of GNNs for representing mathematical structures. Unlike the fixed-length prefix sequences used in this work, GNNs operate directly on expression trees of arbitrary size, naturally accommodating the variability in sub-expression depth across an integration tree and represents a promising direction for future exploration for our problem.

(Suda, 2025) trained a neural clause-selection policy for the Vampire theorem prover using a REINFORCE-style policy gradient method, reporting a 20% improvement over Vampire’s default strategy on the TPTP benchmark. The design adopts two practices also explored in this work: outcome-dependent reward scaling to give each solved problem a fair influence on network updates, and deterministic ranked selection at inference rather than stochastic sampling, on the grounds that ranking is more practical and stable than categorical sampling when the action space contains many low-value options.

Chapter 3

Technical background

3.1 Reinforcement Learning Foundations

To formalise the rule-selection process as a sequential decision-making problem, we use the Markov Decision Process (MDP), an established framework which underpins modern Reinforcement Learning (RL) (Sutton and Barto, 2018).

An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$. At each discrete time step t , the agent observes a state $s_t \in \mathcal{S}$ and selects an action $a_t \in \mathcal{A}$ according to its policy distribution $\pi(a_t | s_t)$. Upon executing the action, the agent receives a scalar reward r_t , and the environment transitions to s_{t+1} according to the transition dynamics $\mathcal{T}(s_{t+1} | s_t, a_t)$. The objective is to find an optimal policy π^* that maximises the expected cumulative discounted return:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^H \gamma^t r_t \right], \quad (3.1)$$

where H is the episode horizon and the discount factor $\gamma \in [0, 1)$ controls the relative weight placed on immediate versus future rewards.

Policy gradient methods directly parameterise the policy as $\pi_{\theta}(a | s)$ and optimise θ via stochastic gradient ascent on J (Williams, 1992). Actor-Critic architectures (Konda and Tsitsiklis, 2000) reduce the high variance of Monte Carlo gradient estimates by maintaining a separately learned value function $V_{\theta_c}(s)$, the *critic*, to baseline the return. The policy, the *actor*, is then updated using the estimated advantage $A(s, a) = Q(s, a) - V(s)$, where $Q(s, a)$ is the expected return from taking action a in state s and then following the current policy.

A further practical difficulty is sensitivity to the step size: a sufficiently large parameter update can collapse policy performance into a poor local optimum. Proximal Policy Optimisation (PPO; Schulman et al. 2017) addresses this by clipping the probability ratio between the updated and old policies within a symmetric interval, yielding the clipped-surrogate objective:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (3.2)$$

where $\rho_t = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and $\varepsilon = 0.2$ is the clipping hyperparameter. By restricting ρ_t to $[1 - \varepsilon, 1 + \varepsilon]$, the objective prevents destructively large parameter updates while permitting multiple gradient epochs over the same rollout data.

Advantage estimates $\hat{A}_t$ are computed using Generalised Advantage Estimation (GAE; Schulman et al. 2016), defined as the discounted sum of one-step temporal difference errors:

$$\hat{A}_t = \sum_{l=0}^{H-t} (\gamma \lambda_{\text{GAE}})^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_{\theta_c}(s_{t+1}) - V_{\theta_c}(s_t), \quad (3.3)$$

where $\lambda_{\text{GAE}} \in [0, 1]$ interpolates between the single-step TD error ($\lambda_{\text{GAE}} = 0$, low variance but biased) and the full Monte Carlo return ($\lambda_{\text{GAE}} = 1$, unbiased but high variance). This work uses $\lambda_{\text{GAE}} = 1$, prioritising unbiased advantage estimates given the bounded episode horizon H .

Finally, an entropy bonus $\beta \mathcal{H}(\pi_\theta)$ is added to the PPO objective to discourage premature convergence to a deterministic policy (Ahmed et al., 2019).

3.2 Sequence Encoders and Mathematical Representation

The neural component of the rule-selection policy must process a symbolic expression and produce a distribution over candidate rules, using the Transformer architecture (Vaswani et al., 2017) adapted for classification via the BERT encoder design (Devlin et al., 2019).

Attention and the Transformer Encoder

The Transformer (Vaswani et al., 2017) computes scaled dot-product attention over query, key, and value projections of the input sequence:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (3.4)$$

where the $1/\sqrt{d_k}$ factor prevents softmax saturation at large embedding dimensions.

Each encoder layer pairs multi-head self-attention with a position-wise feed-forward sublayer, connected via residual additions and layer normalisation following the Post-LN convention of (Vaswani et al., 2017), in which normalisation is applied after each residual sum.

BERT (Devlin et al., 2019) adapts the Transformer encoder for classification by prepending a special [CLS] token to every input sequence. Its final hidden state $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$ aggregates information from all positions via self-attention, providing a fixed-length representation passed directly to a downstream task head. In this work, $\mathbf{h}_{[\text{CLS}]}$ is fed to both the policy head and the value head.

Positional Encoding

Self-attention is permutation-invariant, so positional information must be injected explicitly. We use the sinusoidal encoding of (Vaswani et al., 2017), which defines each dimension of the encoding as a sinusoid of a distinct frequency:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right). \quad (3.5)$$

Tokenisation and Prefix Notation

Standard NLP tokenisation. In natural language processing, vocabulary-construction algorithms such as Byte Pair Encoding (BPE; Sennrich, Haddow and Birch 2016), WordPiece (Wu et al., 2016), and Unigram Language Model (Kudo, 2018) merge or split character sequences to form a subword vocabulary of approximately 30,000 tokens. These methods balance coverage of unseen words against embedding-table size. For mathematical expressions, however, such approaches are inappropriate: operator names (`sin`, `exp`, `pow`) are single meaningful symbols that should never be split, and the vocabulary of elementary-function integration is inherently small and closed.

Mathematical expression tokenisation. Following (Lample and Charton, 2020), the tokenisation scheme treats each mathematical operator as a single indivisible token, each variable as a single token, and each integer coefficient as a sequence of signed digit tokens. Formally, the vocabulary consists of:

- **Operators:** `add`, `mul`, `sub`, `div`, `pow`, `exp`, `log`, `sin`, `cos`, `tan`, and their inverses and hyperbolic counterparts;
- **Variables:** `x` (and `y`, `z` for multivariate extensions);
- **Integer literals:** digit tokens 0–9 with sign prefixes `INT+` and `INT-`;
- **Special tokens:** `[CLS]`, padding token `[PAD]`.

This yields a vocabulary of 40 tokens, two to three orders of magnitude smaller than a typical NLP vocabulary. The small, closed vocabulary eliminates out-of-vocabulary issues for any elementary-function expression and produces a more uniform token distribution, which benefits both training stability and attention pattern interpretability.

Prefix (Polish) notation. An expression tree can be serialised in three natural orders: prefix (operator before operands), infix (operator between operands), and postfix (operator after operands). Infix notation requires explicit parentheses to encode operator precedence unambiguously. Prefix notation places the operator before its operands. Given that each operator in the expression grammar has a fixed arity (e.g., `add` takes exactly two arguments, `sin` takes one), prefix notation is unambiguous without parentheses: a stack-based reader reconstructs the tree in linear time.

Worked example. Consider $f(x) = 3x^2 + \cos(2x) - 1$.

In fully parenthesised infix notation:

$$(((3 * (x ^ 2)) + \cos((2 * x))) + (- 1))$$

In prefix notation using the scheme above:

```
add add mul INT+ 3 pow x INT+ 2 cos mul INT+ 2 x INT- 1
```

Chapter 4

Methodology

4.1 MDP Formulation for Rule Selection

SymPy’s `manualintegrate` process is cast as an MDP as defined in Section 3.1. Integration proceeds recursively: a rule application may decompose the current integrand into one or more sub-expressions, each of which must be integrated in turn. The MDP is defined at the level of individual sub-expressions rather than the full integration tree, so the agent makes one decision per sub-expression node.

State space $\mathcal{S}$. The state $s_t \in \mathcal{S}$ at step t is the current integrand sub-expression requiring integration, represented as a prefix-notation token sequence of length at most 384 tokens (see Section 3.2).

Action space $\mathcal{A}$. The action $a_t \in \mathcal{A}$ is an index in $\{1, \dots, 24\}$, selecting one of the 24 integration rules available in `manualintegrate` (Appendix A, Table A.4). Only a subset of rules is applicable at any given state; rules whose validity conditions are not satisfied return a failure when attempted, incurring a penalty.

Transition function $\mathcal{T}$. Applying rule a_t to state s_t triggers SymPy’s symbolic engine. A successful application either resolves s_t directly to a closed-form antiderivative (terminal) or decomposes it into one or more sub-expressions $s_{t+1}^{(1)}, \dots, s_{t+1}^{(k)}$, each of which become subsequent state to be integrated. If the rule’s applicability check fails, the state is unchanged and the next rule in the ranked list is attempted.

Reward function $\mathcal{R}$. Rewards are assigned per rule attempt. Each attempt that applies a rule validly incurs a step penalty of $r_t = -0.05$; each attempt that fails the rule guard incurs a larger penalty $r_t = -0.2$. The larger failed-attempt penalty reflects that failed guards still consume symbolic evaluation time. After the episode terminates, the step rewards from valid applications are post-scaled by a factor $\lambda(o_T)$ depending on the terminal outcome o_T :

$$\tilde{r}_t = \begin{cases} \lambda(o_T) \cdot r_t & \text{if } r_t = -0.05 \text{ (valid application),} \\ r_t & \text{if } r_t = -0.2 \text{ (failed application),} \end{cases} \quad (4.1)$$

$$\lambda(o_T) = \begin{cases} -15 & \text{correct antiderivative,} \\ +15 & \text{incorrect antiderivative,} \\ +10 & \text{DontKnowRule or error.} \end{cases} \quad (4.2)$$

The post-scaling transforms the effective reward for valid steps on a correct path to $+0.75$, on an incorrect path to -0.75 , and on an indeterminate path to -0.5 . Failed-attempt penalties remain -0.2 regardless of outcome, since failed rule attempts contribute no useful computation to the integration. This design discourages the agent from producing short but incorrect integrals (which are penalised more heavily than long correct derivations) while encouraging correctness even at the cost of additional rule attempts.

Horizon and discount. Episodes are capped at $H = 800$ rule attempts to prevent infinite loops that can arise from cyclic rule interactions in SymPy and a discount factor of $\gamma = 0.99$ is applied.

State sufficiency Each rule in $\mathcal{A}$ has applicability conditions that depend only on the syntactic structure of the current sub-expression s_t . The result of applying a rule is a function of s_t alone, not of the path by which s_t was reached. The transition function $\mathcal{T}(s_{t+1} \mid s_t, a_t)$ therefore satisfies the Markov property.

Transition determinism Given s_t and a_t , the outcome of applying rule a_t to s_t is fully determined by SymPy’s symbolic engine. No stochasticity is introduced by the transition function; all randomness in the training process comes from the policy’s action distribution and from the sampling of training integrands.

Reward boundedness The post-scaled rewards lie in the finite set $\{-0.75, -0.5, -0.2, +0.75\}$. With $\gamma < 1$ and horizon $H < \infty$, the expected discounted return is bounded for any policy π , satisfying the regularity conditions required by the policy-gradient theorem (Sutton and Barto, 2018).

4.2 The Supervised Rule-Selection Model

4.2.1 Architecture

The supervised model is an encoder-only Transformer (Vaswani et al., 2017) following the BERT design (Devlin et al., 2019), with sinusoidal positional encodings and dropout regularisation applied throughout (full configuration in Appendix A, Table A.1). A single linear classification head maps the final [CLS] hidden state to a distribution over the rule action space:

$$\pi_\theta(\cdot \mid s) = \text{softmax}(\mathbf{W}_\pi \mathbf{h}_{[\text{CLS}]} + \mathbf{b}_\pi), \quad (4.3)$$

where $\mathbf{W}_\pi$ is the classification weight matrix and $\mathbf{b}_\pi$ is the bias vector.

4.2.2 Training Data: SIRD

The supervised model is trained on the Symbolic Integration Rules Dataset (SIRD; Sharma, Nagpal and Balin 2023), which contains approximately 2,000,000 labelled pairs (s, a^*) , where s is an integrand sub-expression and a^* is the rule that `manualintegrate` applied at that state during a successful integration trace. The dataset was produced by running `manualintegrate`

over FWD-generated integrands and recording each rule application, yielding one label per decision node in the successful integration tree.

4.2.3 Training Objective

The supervised model is trained by minimising the categorical cross-entropy between the predicted rule distribution and the SIRD label:

$$\mathcal{L}_{\text{sup}}(\theta) = -\frac{1}{N} \sum_{(s, a^*) \in \mathcal{D}_{\text{SIRD}}} \log \pi_{\theta}(a^* | s), \quad (4.4)$$

Full training configuration details are provided in Appendix A.

4.2.4 Inference

At evaluation time, the model produces a probability distribution over the 24 rules for the current sub-expression. Rules are attempted in descending probability order, as described in Section 4.4.1. The inference procedure is identical to that of the RL agent; only the model that produces the probability distribution differs.

4.3 The Reinforcement Learning Agent

4.3.1 Architecture

The RL agent consists of two Transformer encoder networks, an actor and a critic, both sharing the same architectural specification as the supervised model (Table A.1) but trained with entirely different objectives.

Actor network. The actor encodes the current sub-expression s_t and applies a linear policy head to the [CLS] representation to produce a distribution over the 24 candidate rules:

$$\pi_{\theta_a}(\cdot | s_t) = \text{softmax}(\mathbf{W}_{\pi} \mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\pi}). \quad (4.5)$$

Critic network. The critic encodes the same sub-expression and applies a linear value head to produce a scalar state-value estimate used for advantage estimation:

$$V_{\theta_c}(s_t) = \mathbf{w}_V^{\top} \mathbf{h}_{[\text{CLS}]} + b_V. \quad (4.6)$$

The critic is not used at inference time. Figure 4.1 illustrates the interaction between the RL agent and the SymPy environment.

4.3.2 Training Algorithm

The RL agent is trained with Proximal Policy Optimisation (Schulman et al., 2017). At each update iteration, rollout data is collected by running integration episodes on the training set. At each decision node, rules are ranked in descending order of policy probability and attempted sequentially as described in Section 4.4.1; the first rule to pass its applicability guard is applied and its log-probability under the current policy is recorded. For each step t in a rollout, the

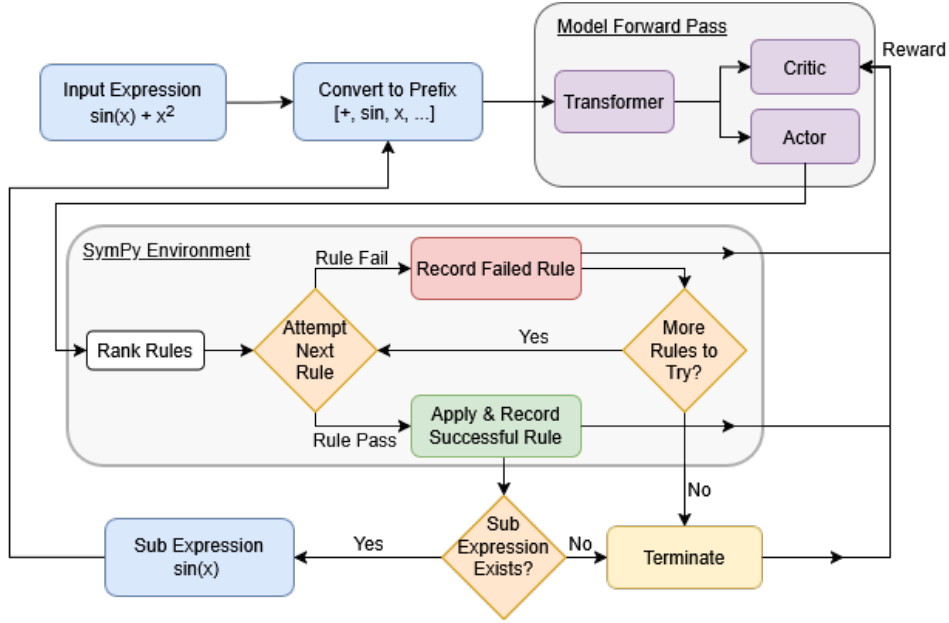


Figure 4.1: Interaction between RL Agent and SymPy environment during training

tuple $(s_t, a_t, \log \pi_{\theta_a}(a_t | s_t), V_{\theta_c}(s_t), \tilde{r}_t)$ is stored, where a_t is the rule applied, $\log \pi_{\theta_a}(a_t | s_t)$ is its log-probability under the current actor, $V_{\theta_c}(s_t)$ is the critic's value estimate, and $\tilde{r}_t$ is the post-scaled reward from Equation (4.1).

Advantage estimation. Advantages are computed using Generalised Advantage Estimation (Schulman et al., 2016) as defined in Equation (3.3), with $\tilde{r}_t$ substituted for the raw reward so that the post-scaled outcome signal propagates directly into the advantage estimates.

PPO objective. The actor and critic are updated jointly by minimising:

$$\mathcal{L}_{\text{PPO}}(\theta_a, \theta_c) = -L^{\text{CLIP}}(\theta_a) + c_v \mathcal{L}^{\text{VF}}(\theta_c) - c_e \mathcal{H}(\pi_{\theta_a}), \quad (4.7)$$

where the three terms are, respectively:

- $L^{\text{CLIP}}(\theta_a)$: the clipped surrogate objective from Section 3.1 (with $\varepsilon = 0.2$), evaluated using the probability ratio $\rho_t = \pi_{\theta_a}(a_t | s_t) / \pi_{\theta_a^{\text{old}}}(a_t | s_t)$;
- $\mathcal{L}^{\text{VF}}(\theta_c)$: the Huber loss between the critic's value estimate $V_{\theta_c}(s_t)$ and the empirical return $\hat{R}_t = \sum_{l=0}^{H-t} \gamma^l \tilde{r}_{t+l}$, with threshold $\delta = 1$;
- $\mathcal{H}(\pi_{\theta_a})$: the policy entropy, included as a bonus to discourage premature convergence to a deterministic policy.

Full hyperparameter values (c_v , c_e , mini-batch size, and update epochs) are given in Appendix A, Table A.3.

4.4 Methodological choices

4.4.1 Ranking Versus Sampling

PPO trains a stochastic policy $\pi_\theta(a \mid s)$ by sampling actions from the distribution and estimating the policy gradient via rollout data. However, directly sampling from π_θ was found to be unstable in early training runs: the vast majority of actions in $\mathcal{A}$ are inapplicable at any given state, and with many consecutive failed samples episodes accumulate large negative returns before a valid rule is found, producing a sparse and noisy training signal.

The approach adopted in this work is to rank rules in descending order of policy probability and attempt them sequentially. The first rule that passes its validity check is applied; if no rule passes, the fallback `DontKnowRule` terminates the episode. This method eliminates repeat failures within a single decision step and was previously used by Suda (2025), who find that a ranked candidate list produces more stable and informative training signals in a related CAS clause-selection environment.

4.4.2 Reward Shaping

The reward structure of Equation (4.1) was designed with three objectives.

First, the per-step valid-application penalty of -0.05 encodes a mild preference toward shorter derivations, discouraging the policy from applying any valid rule regardless of its downstream utility.

Second, the failed-rule penalty of -0.2 is four times larger than the valid-application penalty, discouraging the agent from attempting rules whose applicability conditions are unlikely to be satisfied for the current sub-expression.

Third, the post-scaling design ensures that credit assignment for a correct or incorrect outcome propagates back to every valid rule application in the episode through the advantage estimator. A correct outcome turns each valid step into a positive contribution ($+0.75$); an incorrect outcome turns each into a penalty (-0.75). Outcome-dependent reward rescaling of this kind has precedent in related RL-for-formal-reasoning work: Suda (2025) similarly applies post-hoc reward scaling to ensure training stability in a saturation-based theorem prover.

4.5 Datasets

4.5.1 Training Data

The RL agent is trained on 50,000 FWD-generated integrands, no rule selection information is required since the agent learns solely from reward signals produced by the `manualintegrate` environment.

The data efficiency of RL is a direct result of the multi-step nature of the learning signal: each integration episode produces a sequence of (state, action, reward) tuples, one per rule attempt, so a single integration of a moderately complex integrand may contribute dozens of training steps.

4.5.2 Training Distributions

FWD (forward generation; Lample and Charton 2020). FWD generates integrable expressions by constructing a random elementary function $f(x)$ using a context-free grammar over mathematical operators, then computing $\int f(x) dx$ symbolically using SymPy. The resulting (integrand, antiderivative) pair is guaranteed to be correct by construction. The FWD generator produces the training distributions for both models: the RL agent’s 50,000 unlabelled integrands and the SIRD dataset (which was produced by running `manualintegrate` on FWD integrands; Section 4.2.2).

SIRD (Sharma, Nagpal and Balin 2023). SIRD provides the labelled training data for the supervised model. It contains approximately 2,000,000 (sub-expression, rule) pairs extracted from `manualintegrate` traces on FWD integrands, with one label per decision node in the successful integration tree.

4.5.3 Evaluation Distributions

Both models are evaluated on four datasets. The FWD held-out set is in-distribution; BWD, IBP, and RUBI are out-of-distribution.

FWD held-out. A subset of FWD-generated integrands not used during training. The RL agent’s training integrands and the FWD evaluation set are drawn from the same generator but are disjoint.

BWD (backward generation; Lample and Charton 2020). BWD constructs training pairs in the opposite direction: a random elementary function $g(x)$ is generated and differentiated to yield $f(x) = g'(x)$, so that $g(x) = \int f(x) dx$ by construction. BWD integrands tend to have a different structural profile from FWD: antiderivatives are as complex as the randomly sampled $g(x)$, whereas FWD antiderivatives are constrained by which integrals SymPy can compute.

IBP (integration by parts; Lample and Charton 2020). IBP generates integrands whose antiderivatives are constructed to require integration by parts, producing a distribution weighted toward integrands of the form $u v'$ where u and v involve product structures.

RUBI (Rich, Scheibe and Abbasi 2018). The Rule-Based Integrator (RUBI) provides an independent test suite of 1,832 integrals drawn from mathematical textbooks and research papers. Unlike the synthetic generators, RUBI integrals reflect the kinds of expressions that appear in real mathematical practice and tests both out-of-distribution generalisation and the limits of our models action space.

4.5.4 Train/Validation/Test Splits

The FWD data is split into a training set (50,000 integrands for the RL agent; the full SIRD for the supervised model) and a held-out test set (used for evaluation only). The supervised model uses a 10% validation split from SIRD for early stopping. BWD, IBP, and RUBI are used for evaluation only.

4.6 Evaluation Protocol

4.6.1 Metrics

Each model is evaluated on four metrics per dataset.

Accuracy. The proportion of test integrands for which the model produces a correct antiderivative. Correctness is verified by differentiating the output and checking symbolic equality with the original integrand using SymPy's `simplify`.

Rule attempts (Steps). The total number of rule attempts per integrand, including both successful and failed applications. Fewer steps indicates a more efficient rule ordering.

Branches used. The number of times a rule selection function is successfully applied — equivalently, the number of non-leaf nodes in the integration tree. This measures structural search efficiency and is directly correlated to the length of the final unevaluated rule tree.

Runtime. The average time in seconds from problem submission to answer or failure. This is the metric most directly relevant to end-user latency, but it conflates the cost of neural inference with the cost of symbolic computation; the rule-attempts metric is therefore reported alongside it as a hardware-agnostic alternative.

4.6.2 Baselines

Three methods are compared: `manualintegrate` with its default hard-coded ordering (the heuristic baseline), the supervised model, and the RL agent. All three use the same SymPy environment and are evaluated on the same integrand instances, allowing direct comparison of the metrics above.

Chapter 5

Implementation and Experimental Setup

This chapter describes the software architecture, training infrastructure, and engineering decisions underlying both learned models. Helper functions and full hyperparameter tables are described in Appendix A, and our code, datasets and models are publicly available at https://github.com/rohitpj/RL_for_Integration.

5.1 Environment Design

5.1.1 PPO Implementation

While many PPO implementations exist, we chose to implement it from scratch due to the unique structure of the RL environment. Libraries such as Stable-Baselines3 expect a standard `gym.Env` `step/reset` interface in which each `step()` call advances the environment by exactly one timestep and returns a single (state, reward, done) tuple; the integration environment does not conform to this structure, as a single call to `rl_step` triggers a recursive search of rules for each sub-expression, accumulating a variable number of (state, action, reward) triples before returning control. A from-scratch implementation avoids this mismatch by collecting the episode buffer directly during recursion and triggering the PPO update once the top-level integration call returns.

5.1.2 Modifying `manualintegrate`

`Manualintegrate` module is used as the backbone of our environment with rule definitions and helper functions being imported directly. This allows rule functions to be applied, rewards to be assigned mid-recursion, and the call sequence to be observed without significantly modifying SymPy internals. We maintain the original rule implementations while replacing the heuristic rule selection function (`integral_steps`) with supervised learning and RL based functions (`transformer_step` and `rl_step` respectively).

5.1.3 Episode Structure

Each of the integration rules is a standalone function that accepts an `IntegralInfo` named tuple and returns either a `Rule` dataclass on success or `None` if its applicability conditions are not satisfied. These functions are collected into a module-level dictionary; rule selection and application are handled by a central dispatcher, `rl_step`, which is called recursively as the integration tree is traversed.

When called on a sub-expression, the dispatcher tokenises the integrand, queries the actor network for a probability distribution over the rule set and ranks them in descending order of policy probability. It then iterates through the ordered candidates: a failed rule (a `None` return) incurs the -0.2 penalty and moves to the next candidate, while a successful application incurs the -0.05 penalty and either resolves the sub-expression or decomposes it into child sub-expressions for further recursion. If all rules are exhausted without success, a fallback `DontKnowRule` terminates the episode. A global episode buffer accumulates one (state, action, reward) triple per rule attempt across the entire recursive integration tree; a moderately complex integrand with several sub-expressions typically produces 10–50 buffer entries per episode.

5.1.4 Timeout Handling

Two independent timeout mechanisms prevent runaway episodes. At the MDP level, a hard cap of 800 total rule attempts per integrand is enforced; once reached, `DontKnowRule` is returned immediately. At the evaluation level, a runtime timeout of 30 seconds per expression is applied and expressions exceeding this are recorded as `DontKnowRule`.

5.2 Supervised Model Training

5.2.1 Data Pipeline

Training data is loaded from SIRD in integrand string-rule label pairs. Token vocabularies are constructed from the training split and is used alongside the fitted label encoder across training, validation, and all evaluation datasets. The dataset is split 90/10 into training and validation subsets using stratified sampling by rule class.

5.2.2 Training and Convergence

The supervised model is trained with AdamW using cross-entropy loss over the rule action space. Full hyperparameter values are given in Appendix A, Table A.3. Training converges stably across 15 epochs and completes in approximately 20 minutes on a single GPU. Per-epoch loss is provided in Appendix A, Figure A.1.

5.3 RL Agent Training

5.3.1 Rollout Collection and Reward Assignment

The training loop selects integrands from the training set and calls the integration environment on each. At each decision node, rules are ranked in descending order of policy probability and

attempted sequentially until one passes its applicability guard, as described in Section 4.4.1.

After the integration episode terminates, the reward post-scaling described in Section 4.4.2 is applied: valid-step rewards are multiplied by $\lambda(o_T) \in \{-15, +15, +10\}$ depending on whether the outcome was a correct antiderivative, an incorrect one, or a DontKnowRule result.

5.3.2 Advantage Estimation and PPO Update

After each episode, discounted returns are accumulated with $\gamma = 0.99$ and z-score normalised across the episode buffer before the update. Advantages are then computed as the normalised return minus the critic’s value estimate (Section 4.3.2).

Full hyperparameter values for PPO are given in Appendix A, Table A.4 and 1000 epoch training curves are shown in Figure A.2.

5.4 Engineering Challenges

5.4.1 Unintended Training Behaviour

Several rules in `manualintegrate` are implemented to be structurally applicable to any expression regardless of mathematical validity. `constant_rule`, for example, rewrites any function $f(x)$ as $f(x) \cdot x$ irrespective of whether $f(x)$ is actually constant, relying on an upfront applicability guard to prevent incorrect application. Without outcome-sensitive reward scaling, the agent rapidly discovered that consistently applying such universally-succeeding rules minimised cumulative step penalties while returning an answer immediately, converging to a policy that produced very short but incorrect integrations. Adding explicit guardrails to these rules could partially address this, but doing so would reintroduce the kind of hand-coded heuristics the project aims to replace. Post-scaling with $\lambda = +15$ on valid steps in incorrect trajectories makes the short-but-incorrect strategy unprofitable without requiring any changes to the rule implementations.

5.4.2 Training Time

The RL agent is substantially slower to train than the supervised model, with each episode requiring multiple SymPy rule evaluations per rule attempt and successful rule applications triggering recursive rule applications on sub-expressions. Compounding this, symbolic computation is CPU-bound and cannot be batched or parallelised across episodes and a single complex integrand can take up to a minute of runtime; the default run of 500 episodes requiring over a GPU-hour. This required separate training runs, which made tracking training progress disjointed and hindered more rigorous analysis of methodological decisions.

Chapter 6

Results and Discussion

We compare three models, SymPy’s `manualintegrate` with its default heuristic ordering (`MANUAL`), the supervised behaviour-cloning model (`SUPERVISED`), and the RL agent (`RL`). All three are evaluated on the same sampled expressions within each dataset under identical timeout conditions (30 seconds per expression for RUBI; no runtime cap on the FWD/IBP/BWD evaluation).

Four metrics are reported per method per dataset: **Accuracy** (percentage of correct antiderivatives, verified by symbolic differentiation); **Steps** (total rule attempts); **Branches** (number of successful rule applications, i.e. non-leaf nodes in the integration tree); and **Average time** in seconds per integrand. Steps and branches measure complementary aspects of search efficiency: steps captures the total work done while branches measures the depth and width of the integration tree that was actually constructed.

6.1 FWD, IBP, and BWD Datasets

Table 6.1 presents our primary comparison across all three datasets. The FWD held-out set is in-distribution for both learned models; IBP and BWD are out-of-distribution.

Table 6.1: Integration results across FWD, IBP, and BWD datasets.

Dataset	Method	Accuracy (%)	Steps	Branches	Avg. time (s)
FWD	MANUAL	95.7	101.5	22.5	0.44
	SUPERVISED	94.8	70.2	10.1	0.22
	RL	96.3	53.9	8.3	0.19
IBP	MANUAL	91.7	142.9	28.4	0.60
	SUPERVISED	92.1	54.8	10.9	0.24
	RL	94.5	51.2	7.5	0.22
BWD	MANUAL	41.3	271.3	41.1	1.05
	SUPERVISED	60.5	177.0	23.4	0.77
	RL	69.7	75.9	9.6	0.33

6.1.1 In-distribution performance: FWD

On the FWD held-out set, both learned models outperform `MANUAL` on the efficiency metrics while RL also achieves marginally higher accuracy (96.3% vs 95.7%). `SUPERVISED` reduces steps by 31% relative to `MANUAL` (70.2 vs 101.5) and branches by 55% (10.1 vs 22.5). RL extends these reductions further: steps fall by 47% relative to `MANUAL` (53.9 vs 101.5) and branches by 63% (8.3 vs 22.5).

6.1.2 Out-of-distribution generalisation: IBP

IBP integrands are constructed by integration-by-parts rather than direct integration, however the resulting integrands length and complexity are comparable to FWD. Both learned models maintain their efficiency advantage over `MANUAL` with minimal degradation: RL uses 51.2 steps on IBP versus 53.9 on FWD, a less-than-5% increase, suggesting the policy is not narrowly adapted to FWD’s expression distribution. `MANUAL`’s step count increases substantially from FWD to IBP (101.5 to 142.9), indicating that the fixed heuristic is particularly poorly matched to IBP integrand structure. RL achieves 94.5% accuracy on IBP, compared to `MANUAL`’s 91.7% and `SUPERVISED`’s 92.1%, continuing its FWD advantage in an out-of-distribution setting.

6.1.3 Out-of-distribution generalisation: BWD

The BWD dataset is the most challenging test. BWD integrands are formed by differentiating arbitrary functions, so antiderivatives are at least as complex as the randomly sampled original function. `MANUAL` achieves only 41.3% accuracy on BWD, with an average of 271.3 steps per integrand, indicating that the default rule ordering is especially poorly suited to this distribution.

Both learned models improve substantially over `MANUAL` on BWD, RL achieves 69.7% accuracy (versus 41.3% manual and 60.5% supervised) while using 75.9 steps (versus 271.3 manual and 177.0 supervised). The RL agent explores the integration tree using 76% fewer branches than `MANUAL` and 59% fewer than `SUPERVISED` on BWD, the largest efficiency gap observed across all three datasets. These results indicate that the RL policy has learned a rule-ordering strategy that is more robust than either the fixed heuristic or the behaviour-cloning model for out-of-distribution integrands.

`SUPERVISED` by contrast degrades more severely under distribution shift, its accuracy gap relative to RL grows from 1.5 percentage points on FWD to 9.2 percentage points on BWD.

6.1.4 Supervised model comparison

The supervised model consistently outperforms `MANUAL` on all metrics, confirming that even a simple learned ordering can avoid attempting unnecessary rule attempts. Despite not optimising any trajectory-level metric, supervised learning is sufficient to learn a rule-ranking function that reduces redundant rule applications by 61% relative to the manual heuristic.

The supervised model’s distinct advantage emerges in its training efficiency: 20–40 GPU-minutes versus hundreds of GPU-hours for the RL agent. Where labelled data is available and the deployment distribution is known to closely match training data, the supervised model’s 20–40 minute training time makes it a practical alternative.

6.2 Branching Efficiency Analysis

Branches (the number of successful rule applications) measure the structural complexity of the integration path produced. A large branch count indicates that the integration required many sub-problem decompositions, each of which must itself be integrated. Minimising branches is directly linked to minimising recursion depth, which reduces the risk of stack overflows and redundant symbolic computations. When unevaluated, results with fewer branches are shorter and more easily readable by humans, a significant advantage when using a technique designed to mimic human integral solving.

Figure 6.1 shows the full distribution of branch counts across the FWD evaluation set for all three methods. RL is tightly concentrated at low branch counts, with a sharp peak near 5–10 and very few integrands requiring more than 20 branches. SUPERVISED has a similar peak but a heavier right tail. MANUAL produces a much flatter, right-skewed distribution, with a significant mass of integrands requiring 30–60 branches.

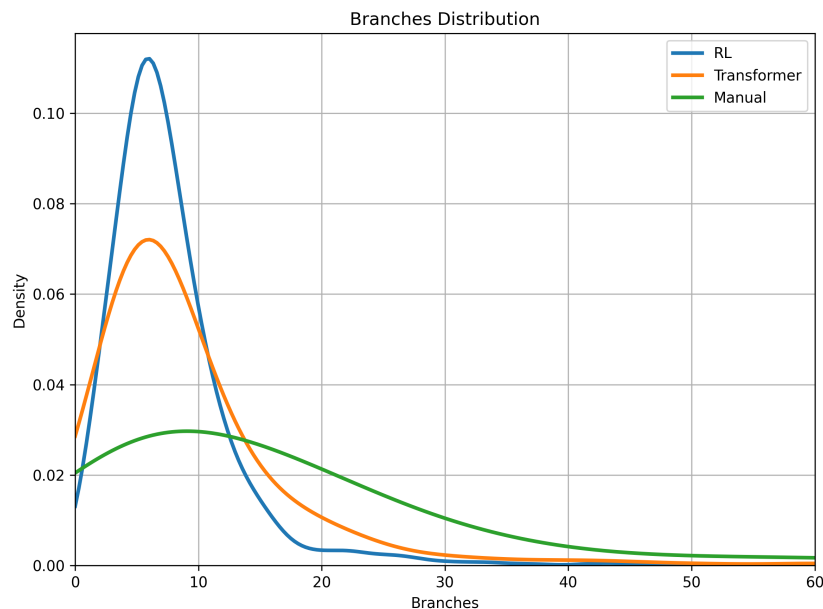


Figure 6.1: Distribution of branch counts on the FWD evaluation set. RL produces a tighter, lower-mass distribution than either SUPERVISED or MANUAL.

Figures 6.2 and 6.3 compare branch distributions pairwise on the same evaluation set. The RL versus MANUAL comparison is most striking: the right tail of the manual distribution — integrands requiring more than 40 branches — is almost entirely absent from the RL distribution, reflecting the agent’s ability to avoid unproductive exploration paths that the heuristic frequently pursues.

The consistent and growing advantage across FWD $\rightarrow$ IBP $\rightarrow$ BWD suggests that the RL policy’s efficiency gains compound under distributional shift — the harder the dataset, the larger the efficiency advantage over the heuristic.

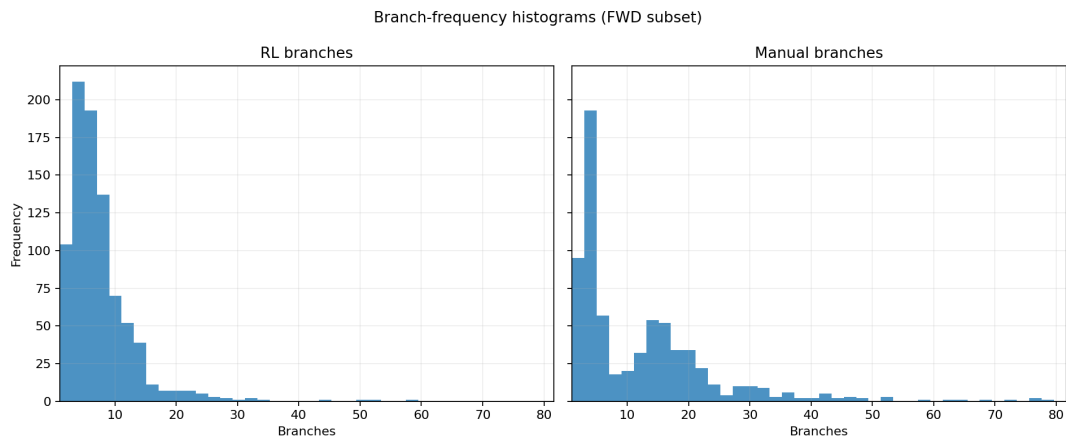


Figure 6.2: Branch-count distributions: RL vs MANUAL on the FWD dataset.

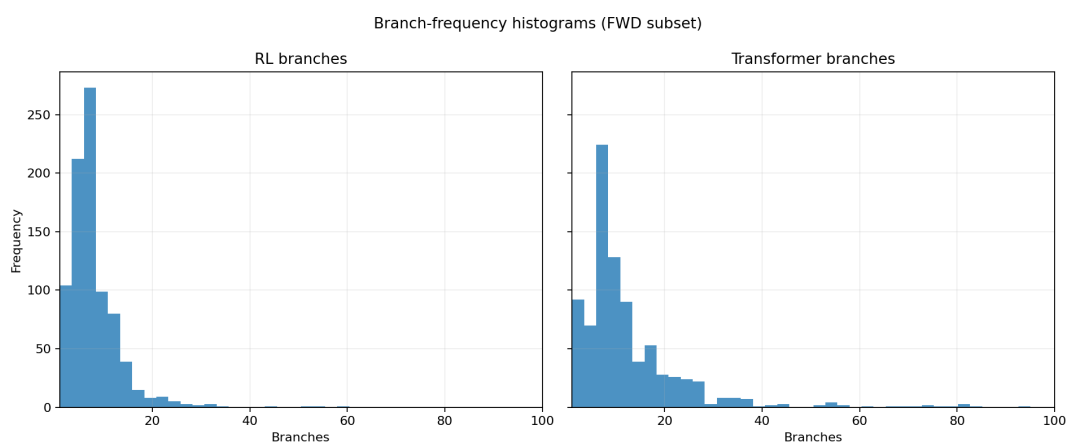


Figure 6.3: Branch-count distributions: RL vs SUPERVISED on the FWD dataset.

6.3 External Validation: RUBI Independent Test Suite

The RUBI test suite (Rich, Scheibe and Abbasi, 2018) provides 1,832 integrals drawn from textbooks and research papers, covering a wide range of function classes. It was constructed independently of the data generators used for training or evaluation above and includes special functions that fall outside the elementary-function vocabulary used during training.

Table 6.2: Performance on the RUBI independent test suite.

Method	Correct (%)	Avg Steps	Avg Branches	Avg Time (s)
MANUAL	39.84	209.62	37.71	1.49
SUPERVISED	43.93	81.52	11.54	0.41
RL	44.43	62.06	8.18	0.30

6.3.1 Performance on RUBI

RL achieves 44.43% accuracy on RUBI, marginally above SUPERVISED (43.93%) and substantially above MANUAL (39.84%) while using 70% fewer steps and 80% shorter runtime

than MANUAL.

This ceiling is expected and is not primarily a failure of the learned policies: many RUBI integrals require rules outside the 24-rule action space, which imposes a hard upper bound on achievable accuracy independent of policy quality.

6.4 Advanced Solving Examples

Table 6.3 presents three integrands for which MANUAL fails to produce a complete antiderivative but RL succeeds. In each case, MANUAL correctly integrates the elementary components but leaves one or more sub-expressions unevaluated, returning a partial result containing a residual integral. RL resolves these residuals by selecting a successful rule at the sub-expression level where MANUAL's heuristic ordering exhausts its budget or makes an ordering error. As seen by these examples, the RL agent has learned a different priority ordering for the `substitution_rule` when encountering nested radical and transcendental function.

Table 6.3: Integrands solved completely by RL but only partially by MANUAL.

Integrand	Manual output	RL output
$x \left(3 + 2x + \frac{x}{\sqrt{x^2 + 1}} \right)$	$\frac{2x^3}{3} + \frac{3x^2}{2} + \int \frac{x^2}{\sqrt{x^2 + 1}} dx$	$\frac{2x^3}{3} + \frac{3x^2}{2} + \frac{x\sqrt{x^2 + 1}}{2} - \frac{\operatorname{asinh}(x)}{2}$
$2x + e^{\sqrt{6}\sqrt{x}}$	$x^2 + \int e^{\sqrt{6}\sqrt{x}} dx$	$\frac{\sqrt{6}\sqrt{x} e^{\sqrt{6}\sqrt{x}}}{3} + x^2 - \frac{e^{\sqrt{6}\sqrt{x}}}{3}$
$\frac{\tan(\sqrt{x})^3 + 2\sqrt{x} + \tan(\sqrt{x})}{\sqrt{x}}$	$2x + \int \frac{\tan \sqrt{x}}{\sqrt{x}} dx + \int \frac{\tan(\sqrt{x})^3}{\sqrt{x}} dx$	$2x + \log(\csc(\sqrt{x})^2 - 1) - 2\log(\cos(\sqrt{x}))$ $- \log(\csc(\sqrt{x})^2) + \frac{1}{\csc(\sqrt{x})^2 - 1}$

Chapter 7

Limitations and Future Work

7.1 Limitations

24 rule action space. The primary ceiling on performance, particularly on RUBI, is the limited rule action space of `manualintegrate`. No learned policy can solve an integral requiring a rule outside this set, and extending the rule set would provide a direct path to improved performance. However this would escalate the exploration problem that arises when most rules are inapplicable at any given state; viable approaches may include hierarchical action selection or per-rule applicability classifiers

FWD training distribution. Both models were trained exclusively on FWD-generated integrands. (Barket, England and Gerhard, 2023) showed that FWD diverges significantly from a uniform sample over integrable elementary functions, and (Welleck et al., 2022) documented structural brittleness in models trained on these distributions. While the BWD and RUBI results demonstrate meaningful out-of-distribution generalisation, training a model on more data sources may significantly improve performance.

7.2 Future Work

Supervised pretraining for RL. A practical limitation of the RL approach is its training cost: hundreds of GPU-hours compared to the supervised model’s 20–40 minutes. Supervised pretraining offers a solution, initialising the actor with the supervised model’s weights would give the RL agent a reasonable rule-ordering prior from the outset. This would reduce the number of episodes required to reach competent performance and allowing subsequent PPO training to focus on improving beyond what the heuristic demonstrator can demonstrate, rather than learning basic rule applicability from scratch. This approach has been shown to reduce total training compute relative to end-to-end RL in formal mathematics settings (Polu and Sutskever, 2020), and both components are already available from this work.

Curriculum learning. The RL agent currently trains on integrands sampled without regard to difficulty, which means early training episodes are dominated by complex expressions the agent cannot yet handle. A curriculum that begins with structurally simple integrands and progressively introduces more complex ones could improve both convergence speed and performance and is straightforward to implement within the existing training loop.

Extending to larger rule vocabularies. Addressing the limited rule coverage ceiling is the highest-impact technical direction. Extending to the full `manualintegrate` rule set or to a subset of RUBI’s 6,700 rules would substantially expand the set of solvable integrals. This requires resolving the exploration problem that arises when the vast majority of actions are inapplicable at any given state; promising approaches include hierarchical action selection (first predicting rule category then specific rule), per-rule applicability classifiers that mask the distribution before the policy acts, or an action-conditioned encoder that scores each rule individually rather than producing a joint distribution over all 24.

Generalising to other CAS algorithms. SymPy’s `dsolve` (ODE solving), `simplify` (expression simplification), and `factor` (polynomial factorisation) all involve heuristic decision sequences structurally analogous to `manualintegrate`’s rule ordering. The environment wrapper, Transformer encoder and PPO training loop developed here can be extended to a new CAS module by updated the action space and reward function for that module. Results in Gröbner basis computation (Peifer, Stillman and Halpern-Leistner, 2020) and cylindrical algebraic decomposition (Jia et al., 2023) demonstrate that similar gains are achievable in related algebraic domains, providing empirical grounds for pursuing this extension.

Chapter 8

Summary and Conclusions

This dissertation investigated whether reinforcement learning can replace the fixed heuristic ordering of `manualintegrate` with a learned rule-selection policy, and whether it can do so more effectively than supervised behaviour cloning on the same architecture. Two learned models were evaluated in a controlled comparison: a supervised Transformer trained on two million state-rule pairs from SIRD, and a PPO actor-critic trained on fifty thousand unlabelled integration episodes. All three hypotheses are supported by the evidence.

H1 — a learned ordering outperforms the heuristic without sacrificing accuracy. The RL agent achieves 69.7% accuracy on the challenging BWD distribution against `MANUAL`'s 41.3%, using 72% fewer steps and 77% fewer branches, and on the RUBI external test suite achieves an 80% reduction in runtime and 77% fewer rule attempts. Efficiency gains are large and consistent; accuracy gains are smaller but present on the out-of-distribution sets.

H2 — RL matches or exceeds supervised learning with substantially fewer labelled examples. The RL agent outperforms `SUPERVISED` on accuracy and efficiency across all four datasets using 40 times fewer training examples and no per-step labels, however the substantially greater training cost is a practical limitation.

H3 — performance gains generalise across integrand distributions. The efficiency advantage grows under distribution shift from FWD to BWD, and the RUBI external test suite confirms that gains extend to real-world textbook integrals with no connection to the training generator.

Our findings show that symbolic integration can be effectively cast as a sequential decision-making problem and improved through RL without sacrificing formal correctness. This design pattern is directly transferable to other heuristic decision sequences in SymPy and in computer algebra more broadly, and the results provide a concrete empirical foundation for pursuing this extension.

Bibliography

- Ahmed, Z., Le Roux, N., Norouzi, M. and Schuurmans, D., 2019. Understanding the impact of entropy on policy optimization. *Proceedings of the 36th international conference on machine learning*. PMLR, *Proceedings of Machine Learning Research*, vol. 97, pp.151–160.
- Bansal, K., Loos, S., Rabe, M., Szegedy, C. and Wilcox, S., 2019. HOList: An environment for machine learning of higher-order theorem proving. *Proceedings of the 36th international conference on machine learning*. PMLR, *Proceedings of Machine Learning Research*, vol. 97, pp.454–463.
- Barket, R., England, M. and Gerhard, J., 2023. Generating elementary integrable expressions [Online]. *Computer algebra in scientific computing*. Springer Nature Switzerland, pp.21–38. Available from: https://doi.org/10.1007/978-3-031-41724-5_2.
- Bronstein, M., 1987. On the parallel Risch algorithm (III). *Proceedings of the 1987 international symposium on symbolic and algebraic computation (issac '87)*. ACM, pp.78–88. Published in proceedings volume dated 1997.
- Bronstein, M., 2005. *Symbolic integration i: Transcendental functions*. 2nd ed. Springer.
- Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding [Online]. *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1*. Association for Computational Linguistics, pp.4171–4186. Available from: <https://doi.org/10.18653/v1/N19-1423>.
- Fitch, J.P., 1981. User-based integration software. *Proceedings of the 1981 acm symposium on symbolic and algebraic computation (symsac '81)*. ACM, pp.139–143.
- Geddes, K.O., Czapor, S.R. and Labahn, G., 1992. *Algorithms for computer algebra* [Online]. Kluwer Academic Publishers. Available from: <https://doi.org/10.1007/b102438>.
- Hubert, T., Mehta, R., Sartran, L., Horváth, M.Z., Žužić, G., Wieser, E., Huang, A., Schrittwieser, J., Schroecker, Y. et al., 2025. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature* [Online]. Available from: <https://doi.org/10.1038/s41586-025-09833-y>.
- Jia, F., Dong, Y., Liu, M., Huang, P., Ma, F. and Zhang, J., 2023. Suggesting variable order for cylindrical algebraic decomposition via reinforcement learning [Online]. *Advances in neural information processing systems*. vol. 36, pp.76098–76119. Available from: https://proceedings.neurips.cc/paper_files/paper/2023/file/efcb5b06ce8bb672ffa26b9dc5cdd0f9-Paper-Conference.pdf.

- Konda, V.R. and Tsitsiklis, J.N., 2000. Actor-critic algorithms. *Advances in neural information processing systems*. vol. 12, pp.1008–1014.
- Kudo, T., 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. *Proceedings of the 56th annual meeting of the association for computational linguistics (acl 2018)*. Association for Computational Linguistics, pp.66–75.
- Lample, G. and Charton, F., 2020. Deep learning for symbolic mathematics [Online]. *International conference on learning representations*. Available from: <https://openreview.net/forum?id=S1eZYeHFDS>.
- Laplace, P.S., 1820. *Théorie analytique des probabilités*. 3rd ed. Courcier. Historical source; cite via Rosenlicht (1972) for modern treatment.
- Meurer, A., Smith, C.P., Paprocki, M., Čertík, O., Kirpichev, S.B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J.K., Singh, S., Rathnayake, T., Vig, S., Granger, B.E., Muller, R.P., Bonazzi, F., Gupta, H., Vats, S., Johansson, F., Pedregosa, F., Curry, M.J., Terrel, A.R., Roučka, v., Saboo, A., Fernando, I., Kulal, S., Cimrman, R. and Scopatz, A., 2017. SymPy: Symbolic computing in Python. *Peerj computer science* [Online], 3, p.e103. Available from: <https://doi.org/10.7717/peerj-cs.103>.
- Norman, A.C. and Moses, J., 1976. The integration of a class of special functions with the Risch algorithm. *Sigsam bulletin*, 11(3), pp.14–17.
- Peifer, D., Stillman, M. and Halpern-Leistner, D., 2020. Learning selection strategies in Buchberger’s algorithm [Online]. *Proceedings of the 37th international conference on machine learning*. PMLR, *Proceedings of Machine Learning Research*, vol. 119, pp.7575–7585. Available from: <https://proceedings.mlr.press/v119/peifer20a.html>.
- Polu, P., Han, J.M., Zheng, K., Baksys, M., Babuschkin, I. and Sutskever, I., 2022. Formal mathematics statement curriculum learning. 2202.01344.
- Polu, P. and Sutskever, I., 2020. Generative language modeling for automated theorem proving. 2009.03393.
- Rich, A., Scheibe, P. and Abbasi, N., 2018. Rule-based integration: An extensive system of symbolic integration rules. *Journal of open source software* [Online], 3(32), p.1073. Available from: <https://doi.org/10.21105/joss.01073>.
- Risch, R.H., 1969. The problem of integration in finite terms. *Transactions of the american mathematical society*, 139, pp.167–189.
- Risch, R.H., 1970. The solution of the problem of integration in finite terms. *Bulletin of the american mathematical society*, 76(3), pp.605–608.
- Rosenlicht, M., 1972. Integration in finite terms. *American mathematical monthly*, 79(9), pp.963–972.
- Schulman, J., Moritz, P., Levine, S., Jordan, M. and Abbeel, P., 2016. High-dimensional continuous control using generalized advantage estimation. *International conference on learning representations*. 1506.02438.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O., 2017. Proximal policy optimization algorithms. *arxiv preprint arxiv:1707.06347*.

- Sennrich, R., Haddow, B. and Birch, A., 2016. Neural machine translation of rare words with subword units. *Proceedings of the 54th annual meeting of the association for computational linguistics (acl 2016)*. Association for Computational Linguistics, pp.1715–1725.
- Sharma, V., Nagpal, A. and Balin, M.F., 2023. SIRD: Symbolic integration rules dataset [Online]. *3rd workshop on mathematical reasoning and AI at NeurIPS*. Available from: <https://openreview.net/forum?id=WWDsbsgyhS>.
- Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Driessche, G. van den, Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T. and Hassabis, D., 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* [Online], 529(7587), pp.484–489. Available from: <https://doi.org/10.1038/nature16961>.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., Driessche, G. van den, Graepel, T. and Hassabis, D., 2017. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. 1712.01815.
- Suda, M., 2025. Efficient neural clause-selection reinforcement [Online]. *Automated deduction – CADE 30*. Springer, *Lecture Notes in Computer Science*, vol. 15825, pp.403–422. Available from: https://doi.org/10.1007/978-3-031-99984-0_22.
- Sutton, R.S. and Barto, A.G., 2018. *Reinforcement learning: An introduction*. 2nd ed. MIT Press.
- Trager, B.M., 1984. *Integration of algebraic functions*. Ph.D. thesis. Massachusetts Institute of Technology.
- Trinh, T.H., Wu, Y., Le, Q.V., He, H. and Luong, T., 2024. Solving olympiad geometry without human demonstrations. *Nature* [Online], 625(7995), pp.476–482. Available from: <https://doi.org/10.1038/s41586-023-06747-5>.
- Ünsal, M., Gehr, T. and Vechev, M., 2024. AlphaIntegrator: Transformer action search for symbolic integration proofs. 2410.02666.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. *Advances in neural information processing systems*. vol. 30, pp.5998–6008.
- Welleck, S., West, P., Cao, J. and Choi, Y., 2022. Symbolic brittleness in sequence models: On systematic generalization in symbolic mathematics [Online]. *Proceedings of the aaai conference on artificial intelligence*. pp.8629–8637. Available from: <https://doi.org/10.1609/aaai.v36i8.20841>.
- Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3–4), pp.229–256.
- Wu, Y., Schuster, M., Chen, Z., Le, Q.V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., Klingner, J., Shah, A., Johnson, M., Liu, X., Kaiser, Ł., Gouws, S., Kato, Y., Kudo, T., Kazawa, H., Stevens, K., Kurian, G., Patil, N., Wang, W., Young, C., Smith, J., Riesa, J., Rudnick, A., Vinyals, O., Corrado, G., Hughes, M. and Dean, J., 2016.

Google's neural machine translation system: Bridging the gap between human and machine translation. *arxiv preprint arxiv:1609.08144*.

Appendix A

Full Hyperparameter Tables and Training Curves

A.1 Software Environment

The implementation uses Python 3 with PyTorch as the sole deep-learning framework and SymPy (Meurer et al., 2017) as the symbolic computation backend. The prefix-notation tokeniser is supplied by the SymbolicMathematics codebase of (Lample and Charton, 2020), whose `CharSPEnvironment.sympy_to_prefix` converts a SymPy expression tree into a flat list of operator/operand tokens.

A.2 Shared Transformer Architecture

Table A.1: Shared Transformer encoder hyperparameters.

Parameter	Value
Hidden dimension d_{model}	512
Attention heads	8
Encoder layers	6
FFN inner dimension	2048
Dropout	0.1
Maximum sequence length	384
Positional encoding	Sinusoidal (fixed)
Layer normalisation order	Post-LN (PyTorch default)

A.3 Supervised Model Training

Table A.2: Supervised model training hyperparameters.

Parameter	Value
Optimiser	AdamW
Learning rate	4×10^{-4}
Batch size	64
Epochs (evaluated)	15
Validation split	10%
Loss function	CrossEntropyLoss
Stratified split	Yes (min 2/class)

A.4 RL Agent Training

Table A.3: RL agent training hyperparameters.

Parameter	Value
Discount factor γ	0.99
PPO clip ε	0.2
Entropy coefficient c_e	0.05
Value loss coefficient c_v	1.0
PPO update epochs	5
Mini-batch size	16
Optimiser (shared)	Adam
Learning rate	10^{-3}
Gradient clipping	None
Max rule attempts H	800
Reward — valid step	-0.05
Reward — failed guard	-0.2
Post-scaling λ (correct)	-15
Post-scaling λ (incorrect)	+15
Post-scaling λ (DontKnow)	+10
Return normalisation	z-score
Advantage normalisation	z-score
Runtime timeout (eval)	30 s

A.5 Training Curves

Figure A.1 — Supervised model convergence

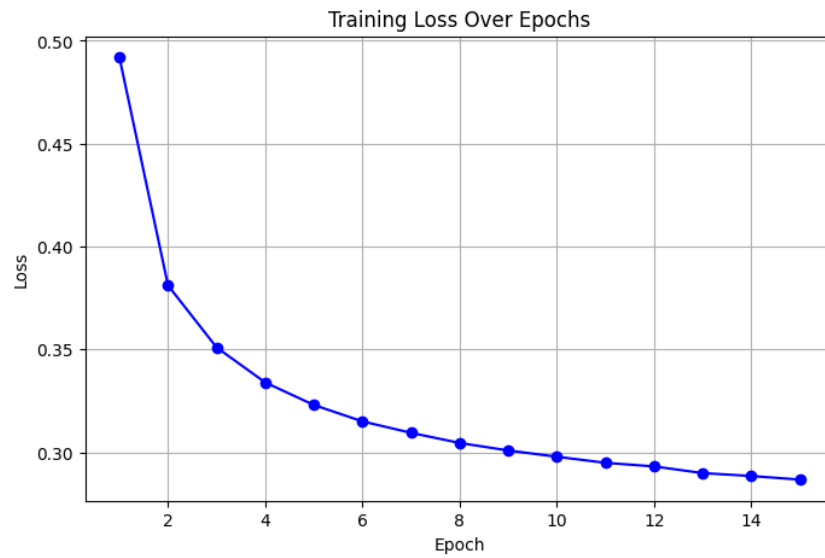


Figure A.1: Supervised training loss over 15 epochs

Figure A.2 — RL agent training curves

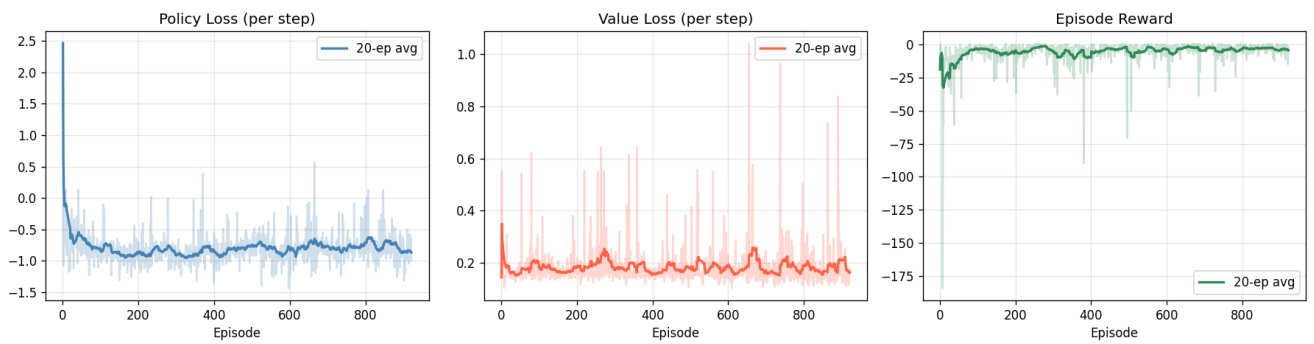


Figure A.2: Policy & value loss & episode reward over 1000 epochs of training

A.6 Integration Rules

Table A.4 provides the integration rules used in the experiment. These are the actions which the RL-agent are allowed to select from. Note that there are more rules other than these ones available, but we opted for the same selection of rules as used in the supervised experiment (?).

Table A.4: Integration rules used by the solver.

Rule Name	Description
add_rule	Integrate sums term-by-term using linearity.
cancel_rule	Simplify rational expressions before integrating.
constant_rule	Integrate constant expressions with respect to the variable.
distribute_expand_rule	Expand products or powers to simpler additive terms.
exp_rule	Integrate exponential functions of the form $e^{f(x)}$.
hyperbolic_rule	Integrate hyperbolic functions (e.g. sinh, cosh).
inverse_trig_rule	Integrate forms producing inverse trigonometric functions.
mul_rule	Handle products, often separating constant factors.
partial_fractions_rule	Decompose rational functions into partial fractions.
parts_rule	Apply integration by parts.
power_rule	Apply the standard power rule $\int x^n dx$.
quadratic_denom_rule	Integrate rational functions with quadratic denominators.
special_function_rule	Recognise integrals producing special functions.
sqrt_linear_rule	Integrate expressions involving $\sqrt{ax + b}$.
sqrt_quadratic_rule	Integrate expressions involving $\sqrt{ax^2 + bx + c}$.
substitution_rule	Perform u -substitution for composite functions.
trig_cotcsc_rule	Integrate powers/products of cot and csc.
trig_expand_rule	Expand trigonometric powers or products.
trig_product_rule	Use identities for products of trig functions.
trig_rule	Integrate basic trigonometric functions.
trig_sincos_rule	Handle products/powers of sin and cos.
trig_sindouble_rule	Apply double-angle identities for sine/cosine.
trig_substitution_rule	Apply trigonometric substitution for radicals.
trig_tansec_rule	Integrate powers/products of tan and sec.